IDS

Final Report

Your Sponsor

Mentor:

Ananth Jillepalli

IDS

Blake Platt, Mitchell Milander, Max Reese

CPT_S 322 Software Engineering Principles I

Semester 2, 2026

TABLE OF CONTENTS

I. Project Outline **4**
I.1. Abstract 4
I.2. Parameters of assignment 4
I.3. Learning Outcomes 4
II. Description and Specifications **5**
II.1. Introduction 5
II.2. Background and Related Work 5
II.3. Project Overview 5
II.4. Client and Stakeholder Information 5
II.5. Team Members and Bios 6
III. Project Requirements Specification **7**
III.1. Project Stakeholders 7
III.2. Use Cases 7
III.3. Functional Requirements 8
III.4. Non-Functional Requirements 8
III.5. Standards 8
IV. Software Design - Solution Approach **10**
IV.1. Architecture Design 10
I. Overview 10
II. Subsystem Decomposition 10
IV.2. Data design 10
IV.3. User Interface Design 10
IV.4. Constraints 10
V. Test Case Specifications and Results **11**
V.1. Testing Overview 11
V.2. Environment Requirements 11
V.3. Test Results 11
VI. Projects and Tools used **12**
VII. Description of Final Prototype **12**
VIII. Social Responsibility and Broader Impacts **13**
IX. Product Delivery Status **13**
X. Conclusions and Future Work **13**

X.1. Limitations and Recommendations 13
X.2. Future Work 14
XI. Acknowledgements 15
XII. Glossary 16
XIII. References 16
XIV. Appendix A – Team Information 17
XV. Appendix B - Example Testing Strategy Reporting 18
XVI. Appendix C - Project Management 18

I. Project Outline

I.1. Abstract

As Cybersecurity focused individuals, we want to create a project which will allow us to apply our interests and gain a greater understanding of how security specialists protect a user's privacy and information. As such, we plan to create a network-based Intrusion Detection System (IDS). This project will allow us to expand our knowledge on cybersecurity goals and principles, while learning to apply the Agile software development process. This experience will act as a baseline which we can build upon as we go through the creation process, making iterations and changes as we progress.

I.2. Parameters of assignment

- A Network-based IDS that captures and analyzes live network packets
- Integration with Npcap for low-level packet capture on Windows
- A desktop graphical user interface built using Qt for real-time visualization
- A system to assess captured packets for malicious behavior
- Continuous Integration using automated testing pipelines

I.3. Learning Outcomes

- Network security and packet analysis
- Python backend development with Scapy
- GUI development using Qt (PyQt/PySide)
- Packet capture using Npcap/libpcap
- Agile development using GitHub
- Continuous Integration using GitHub Actions

- Real-time system design and event-driven programming
- Debugging multithreaded applications

II. Description and Specifications

II.1. Introduction

Given the team's background in cybersecurity and that Intrusion Detection Systems are core to modern computer security, the development team determined that building an IDS is appropriate for the scope of their semester-long software engineering class. Developing an IDS will teach the team about network communication, packet sniffing, and network reporting, while also strengthening their collaborative skills through the use of GitHub for version control and issue tracking.

II.2. Background and Related Work

According to FurtuneBusinessInsights.com, the expected market share of IDS systems is projected to increase by 75% over the next eight years. This comes on the heels of increased investments in cybersecurity by both corporations and government entities as well as advancements in Artificial Intelligence-related IDS solutions. Additionally, IDS systems play a large role in ensuring organizations meet regulatory requirements. This increased reliance on IDS systems nationwide reflects their importance in protecting networks.

Given that Network-based IDS systems are more prevalent in industry than Host-based IDS systems, this project will yield strong educational value for the development team.

Existing Apps for Reference:

- https://github.com/Abrar2652/Network-Intrusion-Detection
- Reddit User's at-Home IDS
- Python IDS

II.3. Project Overview

This project implements a network-based Intrusion Detection System (IDS) with a real-time graphical interface. The system captures live network traffic using Npcap and processes packets through a modular pipeline of connected systems having data sent from packet capture to traffic analysis, threat detection, and lastly alert generation.

The backend is implemented in Python for packet analysis and detection. Captured packets from Qt are processed to extract relevant information such as IP address, ports, packet size, and TCP flags. A detection engine evaluates this data using rule-based logic to identify suspicious behavior.

The application front-end is built using Qt, creating a desktop application which allows users to view packet data, inspect alerts, and monitor network activity. The interface takes inspiration from tools like Wireshark by displaying packet lists and information.

II.4. Client and Stakeholder Information

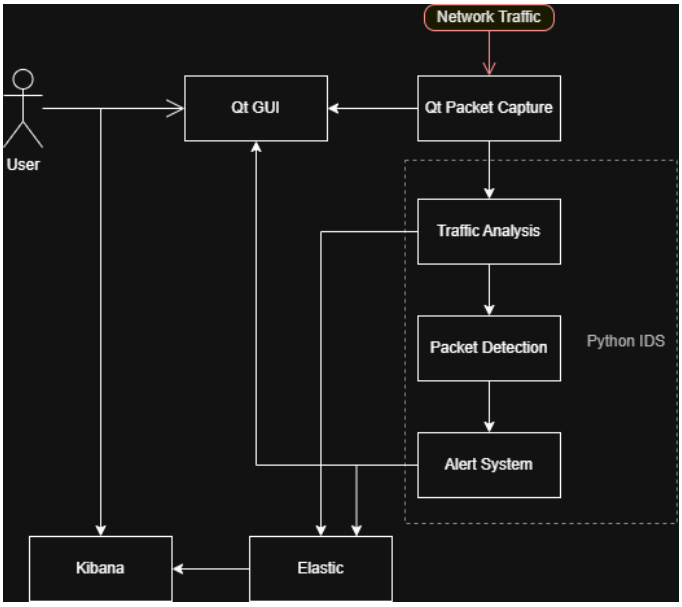
While this project is currently in the scope of academia, potential stakeholders for the project include WSU Cybersecurity Group (for testing and project demos) and Dr. Assefaw Gebremedhin, an associate professor, who could potentially use the IDS to protect systems inside his research lab. One final stakeholder would be the development team (Blake, Max, and Mitchell) who may reuse this IDS for their Senior Capstone project to expand, maintain, and adapt the software.

II.5. Team Members and Bios

1) Blake Platt is a sophomore at Washington State University, pursuing a Bachelor of Cybersecurity. Blakes's Skills are Java, C++, C, and HTML. Currently he is a part of the VICEROY Scholar program and the WSU CYSER program working on a Cybersecurity Undergraduate Research project.
2) Maxwell Reese is a sophomore at Washington State University, pursuing a Bachelors of Cybersecurity. Max has skills in coding with languages such as C, C++, and Java as well as experience with many cybersecurity principles like penetration testing, application security, and packet analysis. He is currently looking for opportunities to get real life experience in the cybersecurity field by going to cybersecurity competitions and searching for related internship positions.
3) Mitchell Milander is a junior at Washington State University, pursuing a Bachelor of Science in Computer Science, with a focus on cybersecurity. The computer languages Mitchell has experience with include C#, Java, C++, and C. Currently, he is a Cybersecurity Undergraduate Researcher in the DoW-funded VICEROY initiative. For this project, he oversees working on the front end of the IDS using React for the GUI and SQL to store alerts, as well as perform statistical analysis and time analysis.

III. Project Requirements Specification

III.1. Use Cases



- **Application Launch**
  User starts the Qt-based IDS application and selects a network interface.
- **Packet Capture (Npcap)**
  Captures raw packets from the network interface.
- **Traffic Analyzer**
  Processes packets that are forwarded from Qt.
- **Packet Detection**
  Determines whether packets are normal or suspicious.
- **Packet Monitoring Dashboard**
  Displays live packet data including IPs, ports, protocol, and size.
- **Alerts Panel**
  Displays detected malicious packets with severity and details.

III.2. Functional Requirements

- Capture live packets using Npcap
- Display packets in real time in a Qt GUI
- Extract packet features (IP, port, protocol, flags)
- Detect suspicious traffic using rule-based detection
- Generate and display alerts in the UI
- Allow user interaction (select packet, inspect details)
- Use test packets to validate detection logic

III.3. Non-Functional Requirements

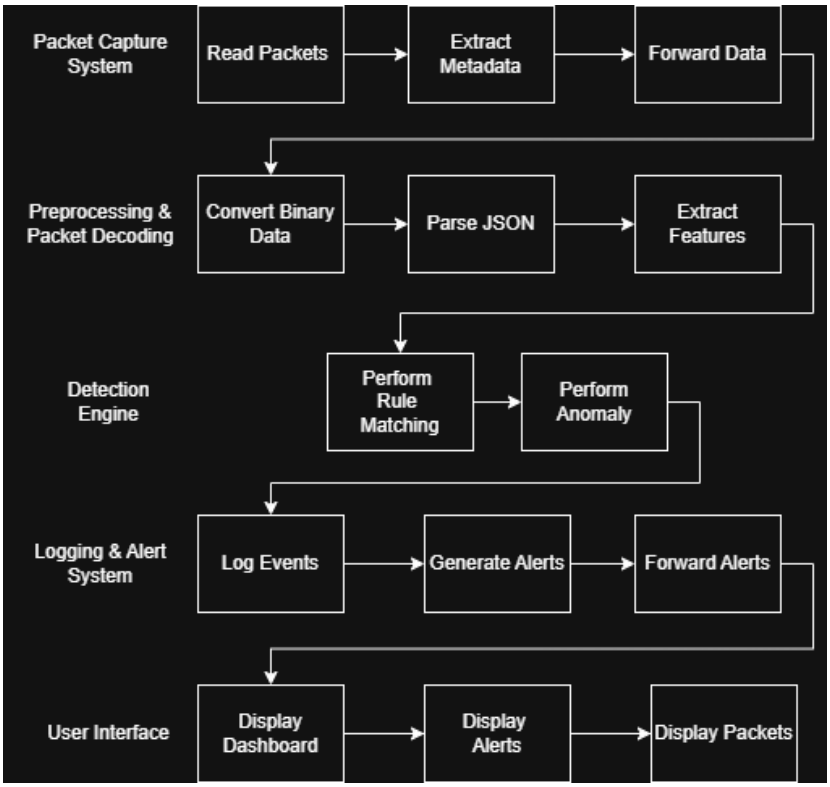
- System must process packets in near real-time
- UI must remain responsive during packet capture
- System should minimize packet loss
- IDS should maintain a low false-positive rate
- Application should run on standard consumer hardware
- Code should be maintainable and modular

III.4. Standards

Here are some standards that govern IDS software. First ISO/IEC 27039 standards focus on integration of intrusion detection and prevention systems or IDPS within an organization's information security management system. This standard aims to enhance operational trust and audit readiness by ensuring that detection and prevention workflows are robust and auditable. The National Institute of Standards and Technology or NIST provides comprehensive guidance on intrusion and prevention systems through its publication which assists organizations in understanding and implementing this technology. This guidance is crucial for organizations looking to establish effective intrusion detection capabilities. Specifically, NIST SP-800-94 helps assist organizations in understanding Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) technologies and in designing, implementing, configuring, securing, monitoring, and maintaining Intrusion Detection and Prevention Systems (IDPS). It provides practical, real-world guidance for each of four classes of IDPS: network-based, wireless, network behavior analysis software, and host-based. The publication also provides an overview of complementary technologies that can detect intrusions, such as security information and event management software. It focuses on enterprise IDPS, but most of the information in the publication is also applicable to standalone and small-scale IDPS deployments. NIST SP-800-31 talks more about the foundational concepts of IDS.

IV. Project Solution Approach

IV.1. Architecture Design



Description: Above is a Block diagram of the IDS architecture design, illustrating the system components and flow of data between them.

I. Overview

The IDS captures packets using Npcap and processes them through a pipeline consisting of feature extraction and threat detection. The detection engine largely evaluates packets against predefined rules to identify malicious behavior.

Processed packet data and alerts are sent to the Qt-based user interface, where they are displayed in real time. The system uses multithreading to ensure that packet capture does not block UI updates

II. Subsystem Decomposition

- Packet Capture System
  - Captures packets from selected interface
  - Uses Npcap backend for Windows compatibility
  - Filters TCP/IP traffic
  - Sends packets to processing queue

- Traffic Analyzer
  - Extracts features from packets
  - Tracks flow statistics (packet count, rate, duration)
- Detection Engine
  - Applies rule-based detection (e.g., SYN flood, port scan)
  - Flags suspicious packets
- User Interface
  - Displays packet list in table format
  - Allows packet inspection
  - Shows alerts in real time

IV.2. Data design

Creating an IDS requires multiple data points from various sources. To organize all data entries, the development team will heavily rely on Elasticsearch which is an open-source search engine for processing data in real time.

Data Components:

- Network Packet Queue
  - Packet Object
    - Timestamp
  - Source IP / Destination IP
  - Source Port / Destination Port
  - Protocol
  - Packet Size
  - TCP Flags
- Flow Data
  - Packet count
  - Byte count
  - Packet rate
  - Flow duration
- Alert Object
  - Timestamp
  - Threat type
  - Confidence
  - Source/Destination info

IV.3. User Interface Design

We use Qt and Elasticsearch for the GUI frontend, and Python for the backend packet analyzer and detector. The Qt GUI shows packets in real time, displays alerts, and allows the user to trigger both a SYN flood and Port scan attack. Packet information includes time, source IP,

destination IP, source port, and destination port. Elasticsearch displays a culmination of data including packet info, alert types, and computer metrics like CPU usage.

IV.4. Constraints

Due to our application running on consumer hardware, the hardware specifications could be a constraint as their device defines the processing power. Another constraint to consider is the number of packets that a consumer's device can process. There is also a constraint on time to consider due to the 2–3-month time limit to complete this software, so some parts of the application are not as polished as they could be given more time. Another constraint is that due to none of us knowing fully how an IDS works, initially, we had to complete tutorials and research more into how one is built so that we can build our own that adds to our time constraints. Furthermore, each of us have other classes that we must juggle creating the need for us to use our time on this project efficiently.

V. Test Case Specifications and Results

V.1. Testing Overview

The goal of testing this Intrusion detection system is to verify that all major components: packet capture, traffic analysis, threat detection, and alerting; function as they should. Our testing will focus on ensuring that packets are processed correctly, that there is proper detection of suspicious activity, and that all activity is correctly visualized.

The test plan includes:

- Verifying packet capture from the selected network interface
- Validating correctness of extracted traffic features
- Testing detection rules (e.g., SYN flood, port scan)
- Ensuring alerts are generated and stored properly
- Confirming data is successfully sent to and visualized in the Elastic Stack

General testing approach:

1. Generate network traffic
2. Capture packets using Scapy and Npcap
3. Analyze packets and extract features
4. Run detection logic (signature-based rules)
5. Generate alerts
6. Store alerts in Elasticsearch
7. Verify results in Kibana dashboard

The objective is to validate that each component works before combining into the full system.

A stretch goal is to use Continuous Integration for testing so that any time a team member pushes his code to GitHub, the unit tests are executed using runners on GitHub action scripts,

for testing the python packet analyzer and Qt packet capturer. Testing tools that will be used include Cppcheck for static analysis of Qt code and pytest for running unit tests on the packet analyzer.

Unit Testing: The IDS has a test function that has been used to test whether the code itself for the IDS works and then as part of other tests. In this test a set number of packets are sent and the output we know. Through this we are able to determine if what we expect to happen does happen.

Integration Testing: Through the already mentioned test function we have been able to test what we want by using what we expect to happen and because of that we can then use this to know what information should be displayed on the Elastic dashboard. This system of using a known value test function to test integrations can be used to verify that the integrations were done correctly.

System Testing: With every component connected: Qt, Python, and Elastic, test packets were sent along with normal traffic. The test packets were correctly identified as suspicious and alerts were sent to Qt. This suggests that the entire system in its final form is correctly functioning.

Functional Testing: We have been doing small functional testing over the course of the development so far. Making sure that each new thing we add is doing what we expect it to do.

Performance Testing: This is not a type of testing we have done yet, but is something we plan to do in the future to test what load our IDS can handle, especially now that there is a separate thread handling POST requests.

Standards and Constraints Testing: Since we are using a GUI, it is our responsibility to abide by the Web Content Accessibility Guidelines (WCAG). The GUI is currently a work in progress. We will use the WCAG guidelines to test that our GUI is Perceivable, by adding text to describe non-text components (Criterion 1.1.1), Operable, by which there are no Down-Events (Criterion 2.5.2), Understandable, by including instructions (Criterion 3.3.2), and Robust, by using HTML in the event we need a markup language (Criterion 4.1.1). For constraints testing, we plan to test the limits of our IDS by sending different size floods of packets and seeing if the IDS gets overwhelmed, and if the Elastic Stack can continue to represent the influx in data.

User Testing: While we have not done everything to test this we have tested the IDS on all of our teams' computers to make sure that it does not only work on one person's computer. Past that we need to now go through what is being shown and make sure it is being shown in a clear way. To obtain user feedback, we plan to have members from WSU's Cybersecurity Club run our IDS and fill out a form that has them assign a rating 1 through 10 on the setup, usability, and functionality of our IDS, as well as a free response feedback section.

V.2. Environment Requirements

Our mode of usage will be client-server to properly examine our IDS and best fit the System-wide testing environment. We will be testing our IDS on two computers with a minimum of 16

GB of ram and the Operating System we are using is Windows 11. Software needed to support the tests are a VPN, Npcap SDK, Elastic Search, Scapy Python module, and Qt.

V.3. Test Results

Aspect Being Tested	Expected Result	Observed Result	Pass or Fail?	Any special configs or libs required?
Packet Capture	Packets captured from interface	Packets successfully captured	Pass	Npcap
Feature Extraction	Correct packet/flow metrics	Values calculated correctly	Pass	Npcap
SYN Flood Detection	High SYN rate triggers alert	Alert Generated	Pass	Scapy
Alert Logging	Alert appears in log file	JSON logs created	Pass	Python json and logging modules
Elastic Integration	Alerts appear in Elasticsearch	Data visible in Kibana	Pass	Elasticsearch Running
Kibana Dashboard	Data visualized correctly	Dashboard displays traffic	Pass	Kibana Configured

V.4. Acceptance Plan

We verified that the system meets the requirements defined in Section III, including real-time packet capture, traffic analysis, threat detection, and alert visualization.

All major system components were tested individually and as part of the integrated system. The IDS successfully detects predefined attack patterns and displays alerts through the Elastic Stack interface.

Therefore, since all functional and system requirements have been satisfied through testing, the project meets the acceptance criteria.

VI. Projects and Tools used

Tool/library/framework	Quick note on what it was for
Visual Studio Code IDE	Used for developing IDS intrusion system.
Qt Creator IDE	Used for implementing Qt interface via the Qt Framework.
Scapy	Used for capturing packets in Python.

Npcap	Used for capturing packets in Qt.
Elasticsearch/Kibana	Used to implement dashboard that displays data graphically.
Docker	Used for running the application on a virtual network.

Languages Used in Project			
C	C++	Python	Batchfile
Qmake	PowerShell	Makefile	

VII. Description of Final Prototype

In its current state, the Windows IDS uses signature-based rule detection to detect both port scan and syn flood attacks on packets, captured by Npcap and optionally Python's Scapy library in real time. After analyzing packets, any alerts are sent to Elasticsearch's Kibana dashboard. Additionally, Qt is used for providing users with a GUI that allows them to select a network interface to listen on and display incoming traffic using Npcap for packet capture. The Python IDS and Qt GUI have also been connected to work together and should be able to provide the user with a one-stop shop for our IDS.

IDS user manual: First go into the IDS deployable folder. Then after you make sure you have all the required software downloaded (Docker, etc) make sure you are connected to the network you would like to start the IDS on. After doing that click on the run_ids.bat file which will start the necessary programs. If issues pop up the startup terminal will show them. After it starts it will open the user interface in your default browser and start capturing packets. In the case that elastic does not open, just run the bat file again and see if that fixes it.

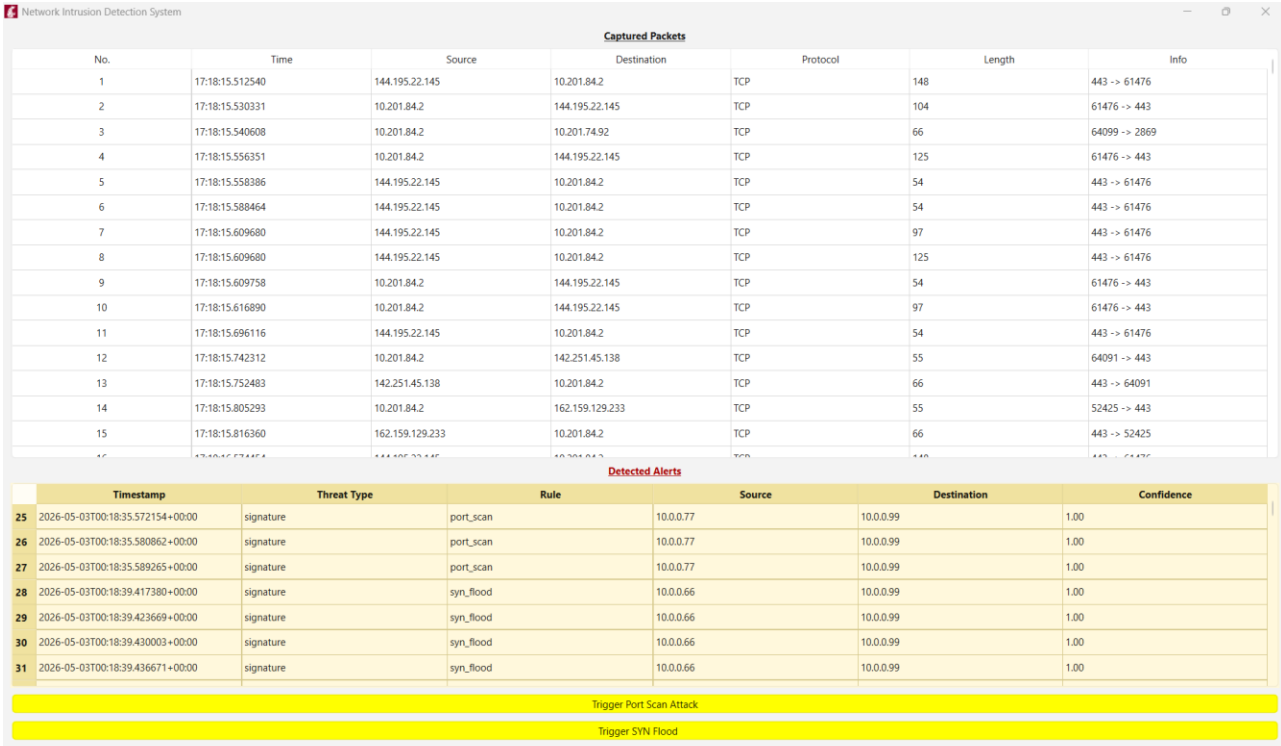
Below is a screenshot of the Kibana IDS Dashboard:

Below is a screenshot of the Python test functions:

```python
def test_ids():
    # Create test packets to simulate various scenarios
    test_packets = [
        # Normal traffic
        IP(src="192.168.1.1", dst="192.168.1.2") / TCP(sport=1234, dport=80, flags="A"),
        IP(src="192.168.1.3", dst="192.168.1.4") / TCP(sport=1235, dport=443, flags="P"),
        IP(src="192.168.1.10", dst="192.168.1.20") / TCP(sport=1500, dport=80, flags="A"),
        IP(src="192.168.1.11", dst="192.168.1.21") / TCP(sport=1501, dport=443, flags="PA"),
        IP(src="192.168.1.12", dst="192.168.1.22") / TCP(sport=1502, dport=8080, flags="FA"),

        # SYN flood simulation
        IP(src="10.0.0.1", dst="192.168.1.2") / TCP(sport=5678, dport=80, flags="S"),
        IP(src="10.0.0.2", dst="192.168.1.2") / TCP(sport=5679, dport=80, flags="S"),
        IP(src="10.0.0.3", dst="192.168.1.2") / TCP(sport=5680, dport=80, flags="S"),

        IP(src="10.0.0.50", dst="192.168.1.2") / TCP(sport=6000, dport=80, flags="S"),
        IP(src="10.0.0.50", dst="192.168.1.2") / TCP(sport=6000, dport=80, flags="S"),
        IP(src="10.0.0.50", dst="192.168.1.2") / TCP(sport=6000, dport=80, flags="S"),

        # Port scan simulation
        IP(src="192.168.1.100", dst="192.168.1.2") / TCP(sport=4321, dport=22, flags="S"),
        IP(src="192.168.1.100", dst="192.168.1.2") / TCP(sport=4321, dport=23, flags="S"),
        IP(src="192.168.1.100", dst="192.168.1.2") / TCP(sport=4321, dport=25, flags="S"),

        IP(src="192.168.1.200", dst="192.168.1.2") / TCP(sport=4000, dport=21, flags="S"),
        IP(src="192.168.1.200", dst="192.168.1.2") / TCP(sport=4000, dport=22, flags="S"),
        IP(src="192.168.1.200", dst="192.168.1.2") / TCP(sport=4000, dport=23, flags="S"),
        IP(src="192.168.1.200", dst="192.168.1.2") / TCP(sport=4000, dport=25, flags="S"),
    ]
```

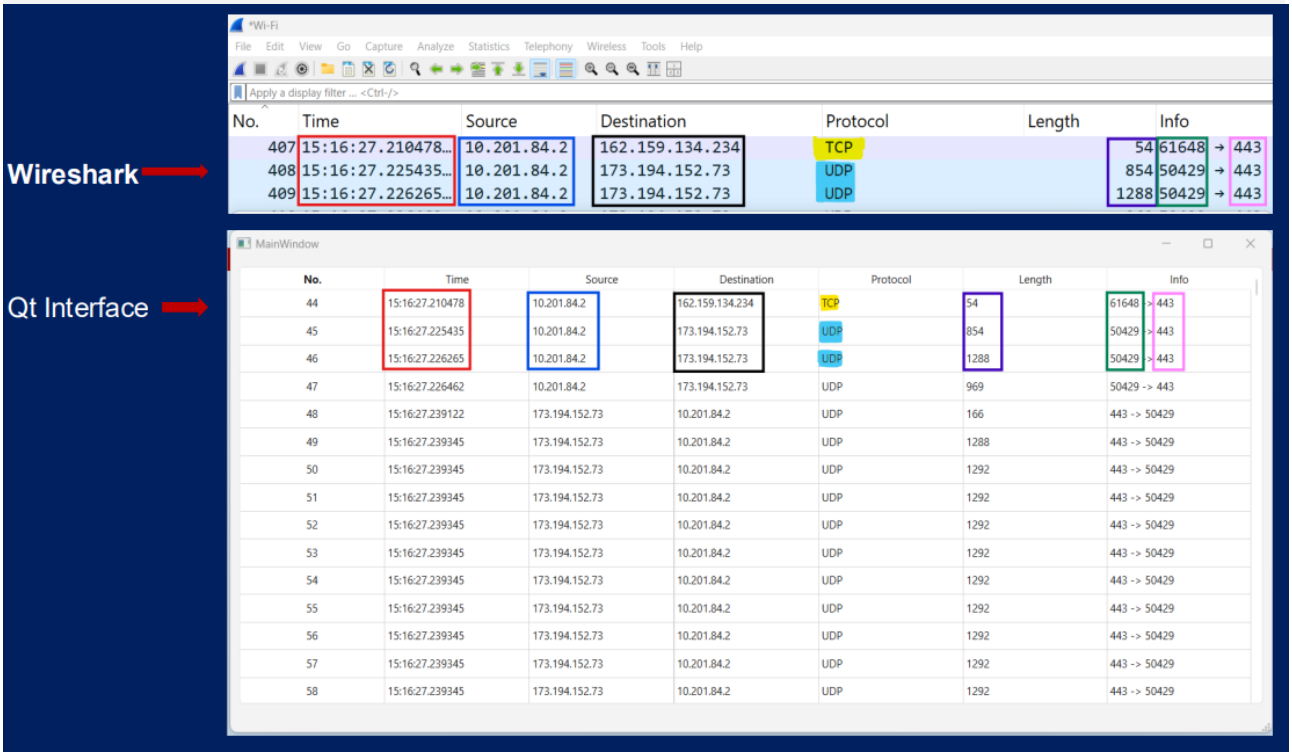
Qt Interface user manual: First cd “IDS Deployable” ./Qt_IDS_Interface and open Qt_Npcap_Integration_Setup markdown file. Follow the instructions and then open/run the Qt_IDS_Interface project file.

Below is a screenshot of the Qt IDS along with a Wireshark test comparison of the packet capturing interface.



No.	Time	Source	Destination	Protocol	Length	Info
1	17:18:15.512540	144.195.22.145	10.201.84.2	TCP	148	443 -> 61476
2	17:18:15.530331	10.201.84.2	144.195.22.145	TCP	104	61476 -> 443
3	17:18:15.540608	10.201.84.2	10.201.74.92	TCP	66	64099 -> 2869
4	17:18:15.556351	10.201.84.2	144.195.22.145	TCP	125	61476 -> 443
5	17:18:15.558386	144.195.22.145	10.201.84.2	TCP	54	443 -> 61476
6	17:18:15.588464	144.195.22.145	10.201.84.2	TCP	54	443 -> 61476
7	17:18:15.609680	144.195.22.145	10.201.84.2	TCP	97	443 -> 61476
8	17:18:15.609680	144.195.22.145	10.201.84.2	TCP	125	443 -> 61476
9	17:18:15.609758	10.201.84.2	144.195.22.145	TCP	54	61476 -> 443
10	17:18:15.616890	10.201.84.2	144.195.22.145	TCP	97	61476 -> 443
11	17:18:15.696116	144.195.22.145	10.201.84.2	TCP	54	443 -> 61476
12	17:18:15.742312	10.201.84.2	142.251.45.138	TCP	55	64091 -> 443
13	17:18:15.752483	142.251.45.138	10.201.84.2	TCP	66	443 -> 64091
14	17:18:15.805293	10.201.84.2	162.159.129.233	TCP	55	52425 -> 443
15	17:18:15.816360	162.159.129.233	10.201.84.2	TCP	66	443 -> 52425

	Timestamp	Threat Type	Rule	Source	Destination	Confidence
25	2026-05-03T00:18:35.572154+00:00	signature	port_scan	10.0.0.77	10.0.0.99	1.00
26	2026-05-03T00:18:35.580862+00:00	signature	port_scan	10.0.0.77	10.0.0.99	1.00
27	2026-05-03T00:18:35.589265+00:00	signature	port_scan	10.0.0.77	10.0.0.99	1.00
28	2026-05-03T00:18:39.417380+00:00	signature	syn_flood	10.0.0.66	10.0.0.99	1.00
29	2026-05-03T00:18:39.423669+00:00	signature	syn_flood	10.0.0.66	10.0.0.99	1.00
30	2026-05-03T00:18:39.430003+00:00	signature	syn_flood	10.0.0.66	10.0.0.99	1.00
31	2026-05-03T00:18:39.436671+00:00	signature	syn_flood	10.0.0.66	10.0.0.99	1.00



No.	Time	Source	Destination	Protocol	Length	Info
407	15:16:27.210478…	10.201.84.2	162.159.134.234	TCP	54	61648 → 443
408	15:16:27.225435…	10.201.84.2	173.194.152.73	UDP	854	50429 → 443
409	15:16:27.226265…	10.201.84.2	173.194.152.73	UDP	1288	50429 → 443

No.	Time	Source	Destination	Protocol	Length	Info
44	15:16:27.210478	10.201.84.2	162.159.134.234	TCP	54	61648 -> 443
45	15:16:27.225435	10.201.84.2	173.194.152.73	UDP	854	50429 -> 443
46	15:16:27.226265	10.201.84.2	173.194.152.73	UDP	1288	50429 -> 443
47	15:16:27.226462	10.201.84.2	173.194.152.73	UDP	969	50429 -> 443
48	15:16:27.239122	173.194.152.73	10.201.84.2	UDP	166	443 -> 50429
49	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1288	443 -> 50429
50	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
51	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
52	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
53	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
54	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
55	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
56	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
57	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429
58	15:16:27.239345	173.194.152.73	10.201.84.2	UDP	1292	443 -> 50429

VIII. Social Responsibility and Broader Impacts

Due to the surveillance nature of our project, we deemed it both necessary and socially responsible to both run and test our IDS in an isolated environment using Docker to containerize our application. This allowed us to simulate port scan and syn flood attacks, safely, without disrupting our apartment complex's own network traffic and possibly setting off their alert systems. In addition, before we integrated the testing process into Qt, we used a VPN to connect to our own hotspot to again keep the IDS on a truly separate network from the public network infrastructure when sending Python test packets. These judgements are in line with our responsibilities as engineers because they first and foremost align with the ISC2 Code of Ethics' first cannon which says to protect infrastructure. It is crucial that the ISC2 Code of Ethics are always applied in any cybersecurity endeavor, and our IDS project is no exception.

One of WSU EECS's goals around benefiting society and broadening participation in STEM is ensuring students gain real world hands-on engineering experience. This is exactly what our team achieved through making this project. Intrusion Detection Systems are used every single day by large and small enterprises to protect their technology systems from incoming malicious traffic. By building our first IDS, we have strengthened our core knowledge around system security, in addition to our technical ability to create a functional detection and alert system that monitors traffic in real time. One example of our project addressing inclusivity is that it follows WCAG Guidelines' Success Criterion 2.5.2 which specifies the avoidance of down-events. For instance, when the user runs the Qt Interface program, they are prompted with a list of network interfaces to choose from and in order to start reading traffic, the user must fully click and depress the mouse pointer on the desired interface for the function to trigger. Given that this project aims to provide users with a Wireshark alternative that incorporates an IDS, there is certainly a foundation for future student-led research and work on our IDS system.

IX. Product Delivery Status

On May 2, 2026, the product will be delivered to our liaison for this project, Instructor Ananth Jillepalli. The product was displayed in front of the class on April 15, 2026. The source repository for this project can be found at this link and setup instructions are found in Section VII of this report.

X. Conclusions and Future Work

X.1. Limitations and Recommendations

The main limitation of the project in its current form is that with the IDS it is hard to test the alert functionality with outside real malicious traffic. As a result, the parameters with which we set to detect malicious traffic has their limitations. The good thing is these parameters can be changed in a simple area of the code if we do gain better insight into real-world attack signatures. We know that the alert functionality works based on changed parameters and the test function, so now more fine tuning is needed to find the range that reliably sends alerts for truly malicious traffic and does not send alerts for normal traffic. Another limitation is right now the dashboard only updates every five seconds. This could possibly be updated to be closer to three or two seconds or maybe even every second but that has not been tested yet.

Another limitation currently is with the Qt Interface that only shows packets coming in, offering no analysis tools for the user like a packet bytes pane, a button to filter packets, and colorization rules. With Npcap and the Qt framework, we are capable of adding that functionality. Currently the packet capture information is at least lining up with that of Wireshark, so any further functionality we implement will hopefully be accurate and that will require some testing.

X.2. Future Work

For future work we would like to expand both the GUI and the functionality of the IDS. Although we got a lot done during this project, there is still room for an enormous amount of growth. If given more time, we would have liked to design the GUI to look even more like Wireshark or a similar tool as that is what we were using for reference for much of the development of this project. We would also have liked to add a Honeypot feature which is designed to intentionally attract malicious traffic, making detection far easier and finding malicious packets more efficient. Lastly, we would have liked to add more detection rules such as adding comparison features which compare the signature of a packet to a known database of malicious packets to streamline detection and make it more secure and reliable.

XI. Acknowledgements

Instructor: Ananth Jillepalli – helped throughout the entire process of the project by helping solidify a plan and providing materials and inspiration for the project.

XII. Glossary

Alert – Notification generated when suspicious or malicious network activity is detected.
Anomaly Detection – Identifies unusual patterns in traffic that may indicate threats, often using machine learning.
Byte Rate – Amount of data (bytes) transmitted per unit of time.
Detection Engine – Component that analyzes traffic and determines if it is malicious.
Flow (Network Flow) – A group of packets sharing attributes (IP, ports) representing a communication session.
GUI (Graphical User Interface) – Visual interface for interacting with the system.
Honeypot – System designed to attract and analyze malicious traffic.
Intrusion Detection System (IDS) – Monitors network traffic to detect suspicious activity.
Machine Learning (ML) – Algorithms used to identify patterns and detect anomalies in data.
Network Packet – Unit of data transmitted over a network containing headers and payload.
Npcap – Windows library for capturing raw network packets.
Packet Capture – Process of intercepting and recording network packets.
Packet Rate – Number of packets transmitted per unit of time.
Protocol – Rules governing data transmission (e.g., TCP, UDP).
Scapy – Python library for capturing and analyzing network packets.
Signature-Based Detection – Detects threats by matching known attack patterns.
SYN Flood – Denial-of-service attack using excessive TCP connection requests.
TCP (Transmission Control Protocol) – Protocol ensuring reliable data transmission.
Threading – Running multiple processes concurrently for performance.
Traffic Analysis – Examining network data to identify patterns or threats.
User Interface (UI) – System component users interact with.
Wireshark – Tool for capturing and analyzing network traffic.

XIII. References

FreeCodeCamp. “Build a Real-Time Intrusion Detection System with Python.” Accessed April 18, 2026. https://www.freecodecamp.org/news/build-a-real-time-intrusion-detection-system-with-python/

Wireshark. *Wireshark Repository*. GitHub. Accessed April 18, 2026. https://github.com/wireshark/wireshark

Abrar2652. *Network Intrusion Detection*. GitHub. Accessed April 18, 2026. https://github.com/Abrar2652/Network-Intrusion-Detection

Coursera. *Introduction to Intrusion Detection Systems (IDS)*. Accessed April 18, 2026. https://www.coursera.org/learn/introduction-to-intrusion-detection-systems-ids

Casado, Martín. *libpcap Tutorial*. Accessed April 18, 2026. https://yuba.stanford.edu/~casado/pcap/section1.html

The Linux Channel. *libpcap Tutorials*. YouTube. Accessed April 18, 2026.
https://www.youtube.com/@TheLinuxChannel/search?query=libpcap

YouTube. *"Building a Simple Client-Server Applet in Qt."* Accessed April 18, 2026.
https://www.youtube.com/watch?v=JacoUUPklHY

Npcap. Npcap Tutorial. Accessed April 18, 2026

https://npcap.com/guide/npcap-tutorial.html

XIV. Appendix A – Team Information

List your team members here and provide a team photo.

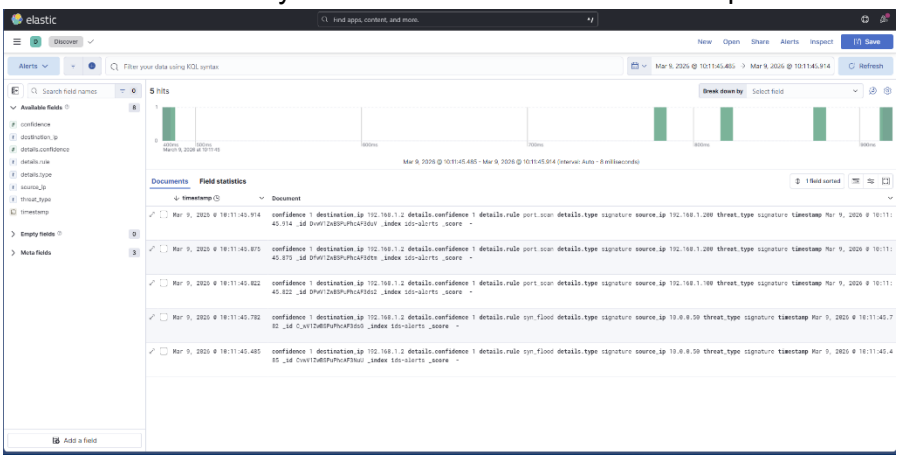
Blake Platt

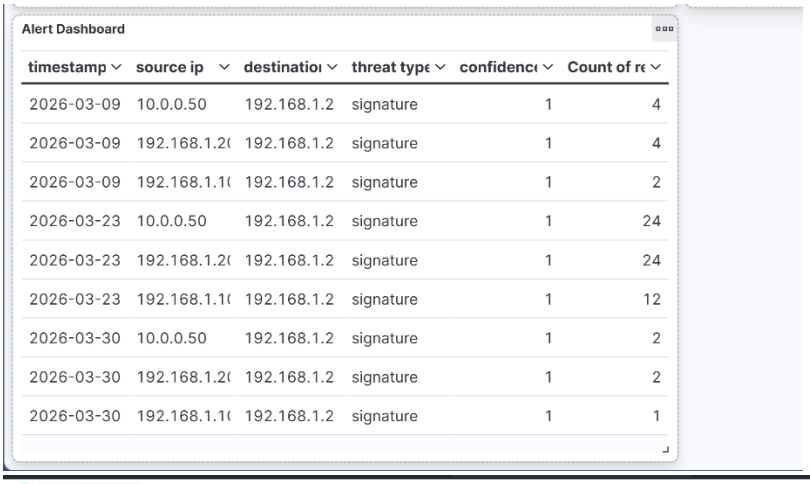
Mitchell Milander

Max Reese

XV. Appendix B - Example Testing Strategy Reporting

1) The requirements that we tested were that packets being captured actually showed up on the UI and that any alerts that were sent also show up on the UI.

2) 



Alert Dashboard

timestamp	source ip	destinatio	threat type	confidenc	Count of re
2026-03-09	10.0.0.50	192.168.1.2	signature	1	4
2026-03-09	192.168.1.2(	192.168.1.2	signature	1	4
2026-03-09	192.168.1.1(	192.168.1.2	signature	1	2
2026-03-23	10.0.0.50	192.168.1.2	signature	1	24
2026-03-23	192.168.1.2(	192.168.1.2	signature	1	24
2026-03-23	192.168.1.1(	192.168.1.2	signature	1	12
2026-03-30	10.0.0.50	192.168.1.2	signature	1	2
2026-03-30	192.168.1.2(	192.168.1.2	signature	1	2
2026-03-30	192.168.1.1(	192.168.1.2	signature	1	1

3) We have not done any user testing at the moment

XVI. Appendix C - Project Management

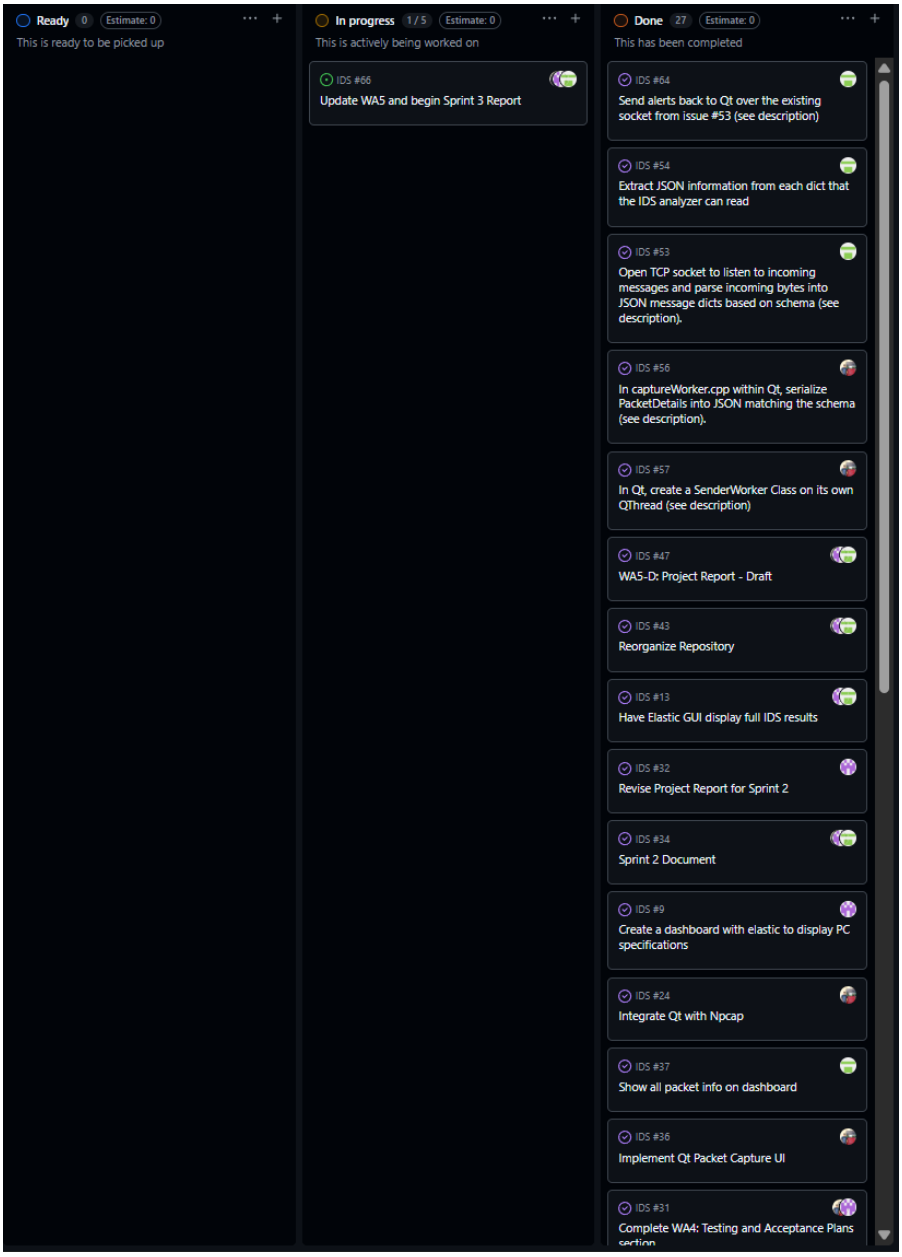
For the mentor meetings, we would first discuss what we did in the last week, including reviewing any submitted documentation for feedback. We would then go over what we were doing for the week and what our plan was.

For the weekly team meetings there was not as much structure. We would mainly talk about what we wanted to get done in the time before the next meeting as well as split responsibilities if needed, i.e. discussing who does what part.

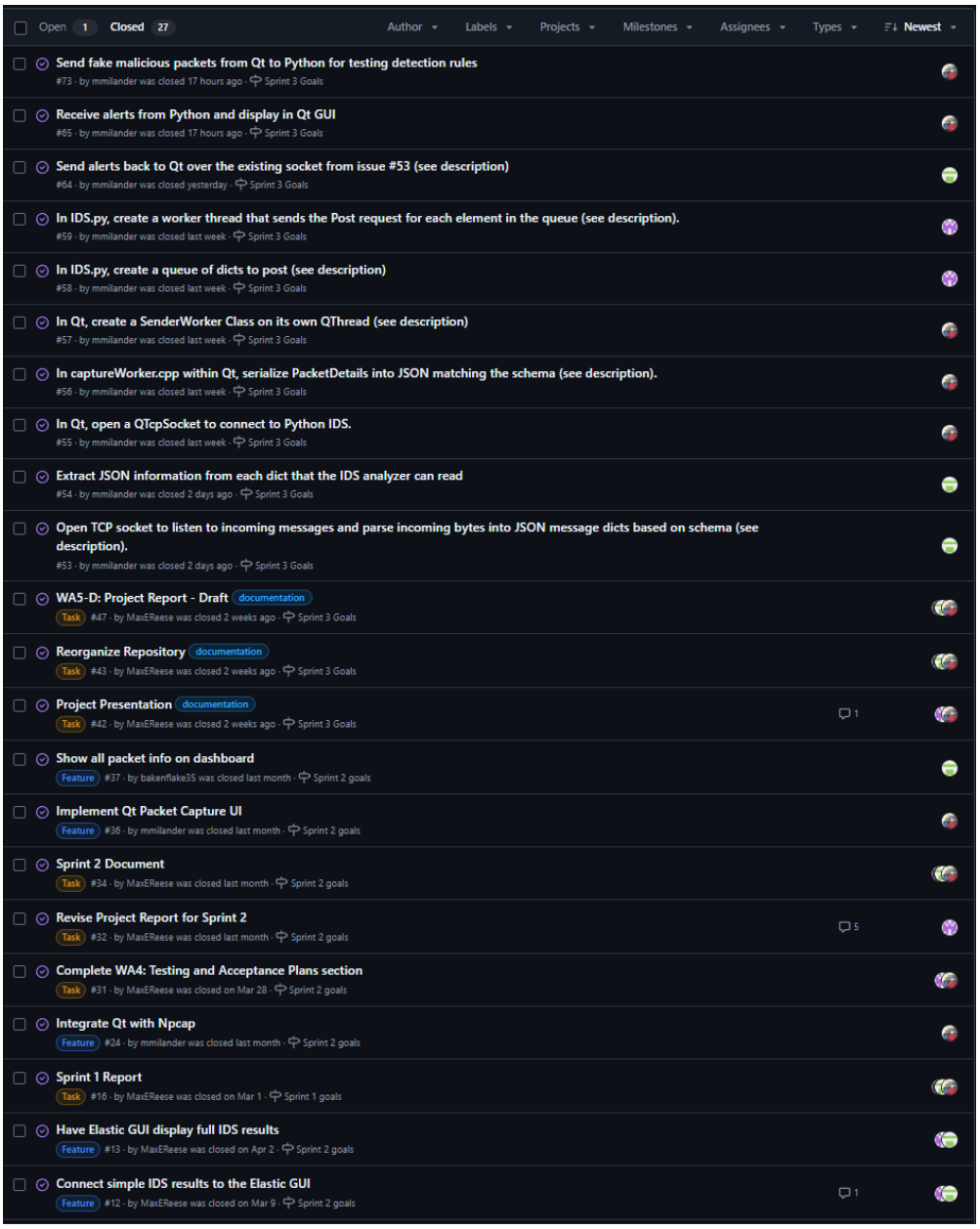
The weekly instructor meetings were very helpful as the instructor would give us feedback on what we did throughout the week, and it would allow us to get a 3rd person perspective on how we did and how we could have done better.

Planning documents:

- GitHub projects - Screenshots & a figure description
    - GitHub projects allowed us to organize our issues and determine what was being worked on by a team member and what needed to be picked up and/or what was completed. It allowed us to have a smooth workflow with little need to ask what needed to be done as it was easily displayed in the projects tab.



- GitHub issues - Screenshots & a figure description
    - We used issues and milestones to help guide progress on what needed to be completed for the current innovation cycle. Whenever a new task needed to be added to the list we would create an issue of which one or multiple people would be able to pick up and start working on. We also used comments on those issues to document progress if it was something more incremental like documentation, for example.



- Notes on team tools used:
  - Email
  - SMS/IM
  - Discord
  - Zoom